SAHARA: Real-Time On-Device Sound Event Detection and Alerting for Deaf and Hard-of-Hearing Users, with India-Specific Acoustic Adaptation

Raj Singh
Dept. of Computer Engineering
MPSTME, SVKM's NMIMS
Mumbai, India

Prachi Teotia
Dept. of Computer Engineering
MPSTME, SVKM's NMIMS
Mumbai, India

Mahi Mayani
Dept. of Computer Engineering
MPSTME, SVKM's NMIMS
Mumbai, India

Advika Mishra
Dept. of Computer Engineering
MPSTME, SVKM's NMIMS
Mumbai, India

Priti Rumao
Dept. of Computer Engineering
MPSTME, SVKM's NMIMS
Mumbai, India
priti.rumao@nmims.edu

***Abstract*—A smoke alarm or a siren is easy to miss for a person who is deaf or hard of hearing (DHH). Few sound-recognition aids report how often they alert for nothing, and almost none have been tried outside Western homes. SAHARA is an Android system that does all of its work on the phone. A frozen YAMNet encoder feeds a small head (296k parameters) that scores eight household sounds, and each alert is a class-specific vibration on the phone and in the firmware of a low-cost ESP32 wristband that we have designed but not yet built. Every class's threshold and temporal rule are chosen together to keep false alarms within a set number per hour, with missed safety sounds counted three times. On held-out public data the model reaches 0.778 mAP and produces 3.5 false alarms per hour on continuous background audio; on the same model, this tuning lowers false alarms from 34.1 to 6.1 per hour. We also recorded SAHARA-IN, 329 sound clips and 107 name-call clips from Mumbai homes and streets. Adding these clips to training lifts cross-validated mAP from 0.621 to 0.822 when folds are random, but gives 0.600 when whole recording sessions are held out. A three-recording home calibration step raises the share of events caught in unseen sessions from 19 % to 35 %. A separate speech model tells the user when someone calls their name.**

***Index Terms*—assistive technology, deaf and hard of hearing, sound event detection, on-device machine learning, haptic feedback**

I. Introduction

The World Health Organization puts the number of people with some hearing loss above 1.5 billion, with more than 430 million of them living with disabling loss [1]. In India, government estimates put the number of people with significant auditory impairment at about 63 million [2]. A smoke alarm, a siren outside, the doorbell, a baby waking up or a relative calling out all carry information that matters, and a DHH person gets none of it unless something turns the sound into light or vibration. DHH users rank safety sounds as most important and want alerts they can read at a glance, wear on the body and keep private [3]–[5].

Embeddings learned on AudioSet [6] have made it realistic to recognize sounds on a phone or a watch [7]–[10]. Three problems remain open. First, most work reports threshold-free scores that treat both kinds of error alike, while the user lives with a stream of alerts. Missing a fire alarm can cost a life, and an aid that buzzes too often is soon switched off [7], [11]. Second, several prototypes send audio or features off the device, which is hard to accept for a microphone that listens all day at home. Third, public corpora are largely Western, and Indian homes sound different, with the whistle of a pressure cooker, temple bells, street vendors calling out and heavy monsoon rain and doorbells that often play a tune.

This paper makes five contributions. (1) Alert rules are tuned to an explicit false-alarm budget. (2) All inference stays on the phone and no audio is ever stored. (3) The app detects when the user's name is called by any voice, and names the caller if that person has enrolled. (4) We collected SAHARA-IN, a field set from Indian homes, test it with whole recording sessions held out, and add a home-calibration step. (5) We give an open wristband design whose Bluetooth protocol and firmware are complete; building the hardware is our next step.

II. Related Work

Sound awareness for DHH users. Matthews et al. compared visual displays of non-speech sound [12], and Bragg et al. designed a mobile sound detector that users could personalize [3]. Findlater et al. surveyed 201 DHH people and found clear interest in sound awareness on phones and wearables [4]; Goodman et al. then tested smartwatch sound feedback in different settings [5]. HomeSound, deployed by Jain et al. in real homes, showed how fast false alerts wear down

users' trust [7], and SoundWatch ran recognition on a watch while studying the balance between accuracy, latency and battery [8]. In a three-week field study, Huang et al. found that background noise, variation within a sound and similar-sounding classes still hold mobile recognition back [11]. On personalization, ProtoSound learns new sounds from a handful of examples using prototypical networks [9], [13], Goodman et al. explored personalization driven by the user [14], and UbiEar offered sound awareness on smartphones that does not depend on location [15]. Earlier wearable prototypes also used smartwatches [16]. Most report accuracy per clip, not alerts per hour of real audio, and were studied in North America or Europe.

Audio embeddings and detection. Transferable sound embeddings are available from several pretrained networks: VGGish [17], OpenL3 [18], PANNs [19], AST [20], and YAMNet [21], an AudioSet-trained MobileNetV1 [22]. Augmentation and weak-label learning are common for small environmental-sound sets [23], [24], and Ubicoustics carried such models over to recognizing everyday activities [10]. We chose YAMNet because it is light enough to run without pause on an Android phone and converts directly to TensorFlow Lite, where integer quantization [25] is also available. We draw domestic events from four public collections [6], [26]–[28], and our metrics follow the reviews by Mesaros et al. [29], [30]. Cost-sensitive training is an old idea [31]; what we add is to keep the unequal costs beyond the loss function, all the way into the rule that decides when to alert.

Keyword spotting and speaker recognition. A small classifier on top of speech embeddings pretrained for keyword spotting can learn a new word from few examples [32]; openWakeWord packages this approach [33]. For telling speakers apart, the GE2E loss yields small embeddings suited to verification [34].

III. System Design

The pipeline is drawn in Fig. 1. Audio is captured at 16 kHz mono by a native Kotlin foreground service, which keeps listening when the screen is off. The samples go into a one-second ring buffer in RAM that moves forward every 0.5 s, and hops whose mean-square energy is below 10^{-4} are skipped as silence. YAMNet, kept frozen, turns each remaining 0.975 s patch into a 1024-dimensional embedding and, in the same pass, gives its own 521 AudioSet scores. We average the two most recent non-silent embeddings and pass the result to a classifier head with two hidden layers (256 and 128 units, ReLU, dropout) and eight sigmoid outputs, 296,328 parameters in total. Each output scores one class on its own: smoke alarm, doorbell, siren, knocking, dog bark, baby cry, glass break or appliance beep. Training uses exactly the window vectors the phone computes (a unit test finds a maximum difference of 0.0), and 54 test clips played through the app's own code raised the same alerts as our evaluation pipeline in all 54 cases.

Privacy. Audio lives only in the ring buffer and is overwritten every half second; nothing is saved to storage, the release build does not request Internet access, and enrollment stores embedding vectors only.

Name calls and open-set notices. If YAMNet's speech score reaches 0.2, the audio also goes to pretrained speech embeddings [32], [33] and a logistic head trained to spot the user's name ("Aryan" in our tests). Next, a GE2E speaker encoder [34] matches the voice against relatives who enrolled once on the phone. At a cosine similarity of 0.70 or higher the alert names the caller ("Raj is calling you"); below that it says "Someone is calling you". A sound outside our eight classes that YAMNet scores at 0.35 or more (running water, say) appears as a silent notice, so it cannot crowd out a safety alert.

Haptics. An alert travels to the wristband as a four-byte packet holding a version, a command, the class and an intensity. Phone and band play the same per-class pattern (Fig. 2). The smoke alarm follows the "temporal-three" rhythm used by evacuation alarms [35], the siren rises and falls, the doorbell is a strong pulse followed by a softer one, and knocking is four short taps. The band (ESP32, coin motor switched by a MOSFET, 500 mAh cell with a protected charger, about Rs. 870 in parts) is designed and its firmware compiles, but it has not been built or measured.

IV. Safety-Weighted Alerting

We treat smoke alarm, siren and glass break as safety-critical and give them a class weight of $w_c = 3$; baby cry and appliance beep get 1.5 and the other classes 1. The head is trained with binary cross-entropy in which only the positive term is weighted, $\mathcal{L} = -\frac{1}{C}\sum_c [w_c y_c \log \hat{y}_c + (1 - y_c)\log(1 - \hat{y}_c)]$, so that missing a positive can cost up to three times as much as a false positive [31]. Per-window sample weights even out the class sizes, and background windows and SAHARA-IN clips are weighted eight and ten times, respectively. We train with Adam at a learning rate of 3×10^{-4} and batch size 128, with early stopping.

Alert rule. Class c raises an alert when its score in the current window satisfies $s_{c,t} \geq \theta_c$, when at least $k_c \in \{1, 2, 3\}$ of its last three windows passed the threshold, and when it has not alerted in the previous 10 s. With window $W = 1$ s, hop $H = 0.5$ s and two pooled embeddings, a sustained sound is reported within $W + 2H + (k_c - 1)H + \tau = 2.0$–$3.0\,\mathrm{s} + \tau$, where τ is the inference time.

Tuning against a budget. We do not fix one threshold for all classes. We run each validation clip through the deployed rule and, for each candidate pair (θ_c, k_c) we record two numbers: the detection rate D_c, the fraction of that class's clips that trigger its alert, and the false-alarm rate A_c, how often per hour it fires on validation audio where the class is absent. We then solve

$$\max_{\{\theta_c, k_c\}} \sum_c w_c \, D_c(\theta_c, k_c) \quad \text{s.t.} \quad \sum_c A_c(\theta_c, k_c) \leq B \qquad (1)$$

by sweeping a Lagrange multiplier, and set the budget to $B = 4$ alerts per hour (Fig. 3). Model selection also looks only at validation data: among runs trained on the current

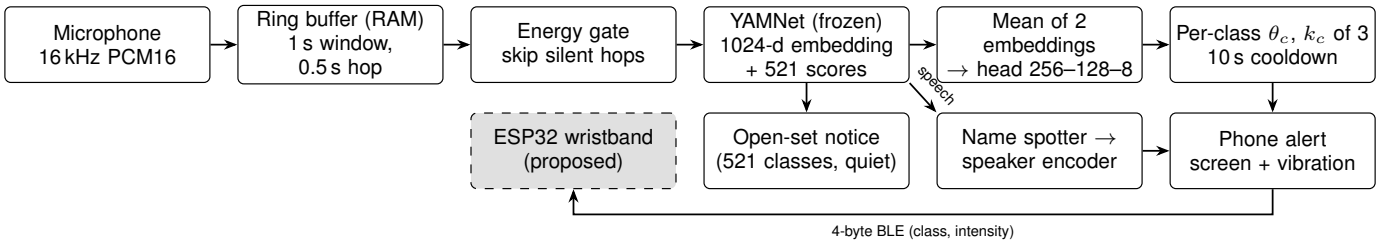


Fig. 1. Processing chain of SAHARA. All stages run inside an Android foreground service on the phone; the Bluetooth link to the planned wristband (dashed) carries nothing but a class code and an intensity value.

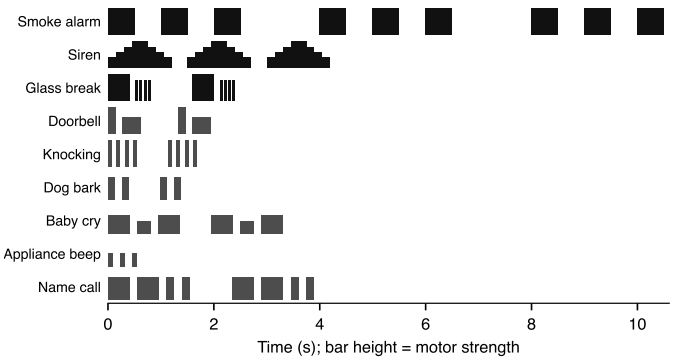


Fig. 2. Per-class vibration patterns taken from the firmware and drawn on a shared time axis. Bar height gives motor strength; the safety classes at the top always vibrate at full strength.

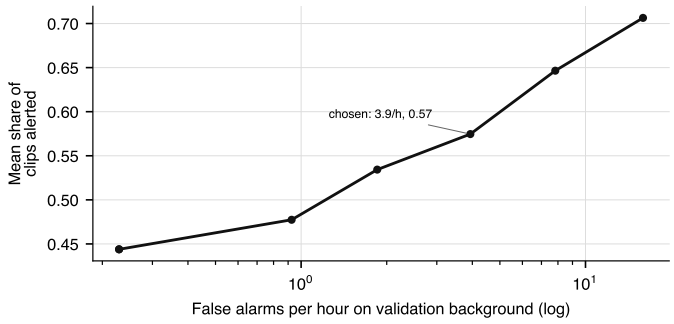


Fig. 3. Trade-off on validation data for one model: average fraction of clips that trigger an alert versus the allowed false alarms per hour (log axis).

data, we keep the one with the best tuned score whose safety classes reach a precision of at least 0.70. Solving it for $B \in \{1, 2, 4, 8, 16\}$ gives the app's five sensitivity steps, each with a measured trade-off.

V. Data and Protocol

Public data. Target sounds are drawn from FSD50K, AudioSet and ESC-50. Background audio comes from 10,000 FSD50K clips that carry no target-like label, from the 44 ESC-50 categories outside our classes, and from DESED soundscapes (Table I). Checking every label identifier against the official vocabularies caught serious mistakes in an earlier build, such as AudioSet "glass" identifiers that pointed to other classes, and FSD50K's generic *Alarm* label mapped to appliance beep. Each class now has at least 150 real clips, with at least 20 of them in the test split. The split is 70/15/15 and is grouped by the source recording (its YouTube or Freesound identifier), so no recording is shared between splits. In addition, we overlay each training target on three backgrounds from the training split at SNRs between -5 and 10 dB, giving 6,834 mixtures; 564 test mixtures made the same way are reported on their own, never pooled with real data.

Processing. All audio is resampled to 16 kHz mono and cut into the same 1 s windows that the phone uses. YAMNet embeds each window once, and the result is cached for 22,521 clips. AudioSet labels are weak (they say a sound occurs somewhere in a clip), so for those clips we keep as positives only the windows where YAMNet itself scores the class highest. The remaining windows of a target clip are discarded rather than treated as background.

SAHARA-IN. Four of us collected 329 clips with our own phones in homes and on streets in Mumbai, over three uploads. Of these, 269 contain target sounds (knocking 67, appliance beep 59, doorbell 58, smoke alarm, siren and glass break 20 apiece, baby cry 14, dog bark 11) and 60 are background (street vendors, a temple, kitchen noise, TV, rain, and doors and drawers). Roughly 40 % of every label is kept for testing. Each upload is split on its own, so later clips never move earlier ones. Another 107 clips, 38 of the name "Aryan" and 69 of similar-sounding names, are used for the name-call model, which we evaluate with each recorder left out in turn. Because the recordings come from only a few sittings, we also assign each clip to a recording session (its recorder, or, for uploads without a recorder name, the phone's numbered file-name series) and report cross-validation that holds out whole sessions.

Mixed sources. The sources differ in device, level, length and labeling (human-verified FSD50K, weak AudioSet labels, curated ESC-50, synthetic DESED, phone recordings). To limit the bias, every clip goes through the phone's resampling, windowing and YAMNet embedding, weak labels are narrowed to the windows holding the event, splits are grouped by source recording, and public, Indian and synthetic results are always reported apart.

Processing timeline. We downloaded the public corpora and built the first manifests in August 2026. In September the team made the first SAHARA-IN upload (recorded on 16–17 September) and we generated the synthetic mixtures. In early

TABLE I
NUMBER OF REAL CLIPS FOR EACH CLASS BY SOURCE. BRACKETS GIVE THE TEST-SPLIT COUNT, SAHARA-IN INCLUDED; † MARKS SAFETY CLASSES. THE BACKGROUND ROW EXCLUDES 605 FURTHER DESED CLIPS.

Class	AudioSet	ESC-50	FSD50K	SAHARA-IN	Total (test)
Smoke alarm†	183	–	–	20	203 (35)
Doorbell	92	–	144	58	294 (57)
Siren†	34	40	132	20	226 (42)
Knocking	97	40	373	67	577 (100)
Dog bark	10	40	536	11	597 (94)
Baby cry	106	40	–	14	160 (30)
Glass break†	97	40	510	20	667 (105)
Appliance beep	261	40	206	59	566 (101)
Background	–	1,760	10,000	60	12,425 (1,876)

TABLE II
PER-CLASS RESULTS ON THE PUBLIC TEST SPLIT WITH THE SHIPPED SETTINGS: THRESHOLD θ_c, REQUIRED HITS k_c OUT OF 3 WINDOWS, WINDOW-LEVEL AP AND F1, AND NUMBER OF CLIPS THAT RAISED THE ALERT.

Class	θ_c	k_c	AP	F1	Clips alerted
Smoke alarm†	0.81	2	0.916	0.858	22/27
Doorbell	0.995	1	0.711	0.432	13/35
Siren†	0.91	1	0.875	0.817	28/34
Knocking	0.95	1	0.604	0.410	21/73
Dog bark	0.98	1	0.908	0.847	67/90
Baby cry	0.99	2	0.817	0.712	17/25
Glass break†	0.93	1	0.827	0.790	56/97
Appliance beep	0.90	3	0.567	0.496	17/78
Mean			0.778	0.670	0.56

October we audited every label identifier, rebuilt the splits so that no source recording leaks across them, cached window-level YAMNet embeddings and added the budget-tuned alert rule. The second and third uploads (95 and 161 clips, the third bringing the team's first smoke alarms, sirens and glass breaks) came on 29–30 October; we then retrained the head and ran the session-held-out and home-calibration experiments reported below.

Metrics. We report average precision (AP) per class over 0.5 s windows and its mean (mAP), together with precision, recall and F1 at the tuned thresholds. We also report the fraction of test clips for which the shipped rule raises the correct alert, and false alarms per hour with all background audio streamed back to back. Thresholds, alert rules and the model itself were chosen on validation data only.

VI. RESULTS

Detection. Table II breaks down the public test results: 0.778 mAP overall and a macro F1 of 0.670. Streaming 3.4 h of test background through the shipped rule gives 3.5 false alarms per hour, half of them for "siren". Smoke alarm and siren do well: 22 of 27 smoke-alarm clips and 28 of 34 siren clips raise an alert. The third safety class, glass break, is weaker, alerting on 56 of 97 clips. Knocking (AP 0.604) and appliance beep (0.567) are the hardest classes. These sounds are short and varied, and Fig. 3 suggests the model, not the alert rule, is the limit. When test clips are mixed into loud background at -5 to 10 dB, mAP drops to 0.666, so heavy background noise is still the main difficulty. Max-pooling instead of averaging the two embeddings did not help short sounds (validation score 0.609 vs. 0.614, five seeds).

Effect of the tuning. We replayed all 2,435 real test clips (4.6 h) through three rules on one model (Table III). A false alarm here is any alert for a class on audio without it, including other classes' clips, so these rates exceed the background-only figure. With a flat 0.5 threshold and 2-of-3 smoothing the phone would buzz falsely 34.1 times an hour, roughly every 105 seconds. With budget tuning this falls to 6.1 per hour, and the model still alerts on 54 % of all clips and 69 % of safety clips. Across the five sensitivity steps, the share of clips alerted goes from 47 % at 2.5 false alarms per hour to 67 % at 20.6.

TABLE III
THREE ALERT RULES APPLIED TO THE SAME TRAINED MODEL ON EVERY REAL TEST CLIP: AVERAGE FRACTION OF CLIPS ALERTED (ALL CLASSES, SAFETY CLASSES) AND FALSE ALARMS PER HOUR.

Alert rule	All	Safety	FA/h
$\theta = 0.5$, 1 window	0.766	0.838	49.1
$\theta = 0.5$, 1 (safety) or 2 of 3 windows	0.688	0.838	34.1
Tuned (θ_c, k_c), $B = 4$/h (SAHARA)	0.538	0.692	6.1

Indian conditions. For SAHARA-IN we used five folds. When folds are drawn at random, training on the team clips lifts mAP from 0.621 to 0.822, and the count of target clips that raise an alert goes from 45 to 100 out of 269. AP for smoke alarms climbs from 0.385 to 0.801 and doorbell AP from 0.521 to 0.824. When whole recording sessions are held out instead, the overall gain is gone (0.621 without the clips, 0.600 with them), though doorbells still improve (0.521 to 0.691). So what the model picks up is mostly the acoustics of the homes it has heard, not Indian sound in general. This also means the SAHARA-IN test split, whose sessions overlap with training, flatters the model (0.831 mAP, versus 0.698 for the model trained before the last upload).

Home calibration. Hence a calibration step: the user records a sound three times in the place where it happens, and the app places that class's threshold half a log-odds unit under the lowest score among the three. It never goes below the threshold of the most permissive (16 per hour) step, and an optional recording of everyday home sound can raise it again. On session-held-out models (three clips of an unseen session for calibration, the rest as events, repeated over 20 random draws), across 17 session–class pairs the share of events caught rises from 19 % to 35 %, at a cost of about one extra false alarm per hour for each calibrated class. Doorbells benefit most (in one home, from 20 % to 83 %). Knocking, glass break and smoke alarm do not improve, because in those sessions their scores stay below even the lowest threshold we allow.

Name calls. With one recorder left out at a time, the spotter tells calls of "Aryan" apart from background scenes at a ROC

AUC of 0.872 (equal error rate 19 %), and from similar names such as Arya and Aryanshi with an AUC of 0.743 (32 %). When every caller is enrolled and the best match is always taken, the encoder names the right person for 29 of 38 calls. At the app's default acceptance threshold of 0.70, 19 of 38 calls from enrolled callers are named correctly and, when the caller is not enrolled, 82 % of calls correctly read "Someone". Since each of us recorded on our own phone and in our own room, these figures partly reflect device and room as well as voice.

On-device cost. We measured the app on a Samsung Galaxy S24 running Android 16. Processing one window costs 20–29 ms with the display on and 47–49 ms with it off, well within the 500 ms hop. Model files total 24.3 MB (YAMNet 15.0, head 1.2, name-call models 8.1), the release APK is 45.8 MB, and the debug build used 128.9 MB of memory. INT8 quantization of the head cost 0.015 mAP (0.765 vs. 0.780), but with thresholds as high as 0.995 small score shifts can flip alerts, so we ship the float model. In a playback test from laptop speakers, smoke alarm, siren and appliance beep alerted within 2.5 s of the sound starting, inside the bound derived above. The other five did not alert, largely because laptop speakers reproduce knocks, barks and doorbells poorly; they still need testing with real sources.

VII. Discussion and Limitations

At the chosen budget of 4 false alarms per hour, the model alerts on 57.5 % of validation events, and allowing four times as many false alarms only lifts this to 70.6 %. Short sounds therefore need a stronger model (e.g., fine-tuning YAMNet's last layers) or more data, not a looser rule. SAHARA-IN is small. It has 20 clips for each safety class, each of those classes was recorded in a single session, and the background test set is only a few minutes long, so claims about unseen Indian homes need many more households. The name spotter knows one name and mistakes similar names for it about a third of the time, and its speech models carry a non-commercial license. The wristband, full-day battery use and a study with DHH participants are still to come; we plan a field deployment in Mumbai that measures missed events, alert fatigue and how well users recognize the vibration patterns [4], [11].

VIII. Conclusion

SAHARA shows that a sound-awareness aid for DHH users can run fully on a phone, keep no audio, and be tuned for the figure that decides whether people leave it switched on: false alarms per hour. With the model held fixed, budget-driven tuning of thresholds and window counts reduced false alarms by a factor of more than five. Recordings from Indian homes helped the model with those particular homes but not yet with homes it had never heard, and a calibration step that needs only three recordings closes part of that gap. A separate speech model lets users know when their name is called and by whom. Our code, tuning scripts, BLE protocol, firmware and the SAHARA-IN recording protocol are public at https://github.com/Rajsingh404/Sahara.

References

[1] World Health Organization, *World Report on Hearing*. Geneva: WHO, 2021.

[2] Directorate General of Health Services, Ministry of Health & Family Welfare, Government of India, "National Programme for Prevention and Control of Deafness (NPPCD)." [Online]. Available: https://dghs.mohfw.gov.in

[3] D. Bragg, N. Huynh, and R. E. Ladner, "A personalizable mobile sound detector app design for deaf and hard-of-hearing users," in *Proc. ACM SIGACCESS Conf. Computers and Accessibility (ASSETS)*, 2016, pp. 3–13.

[4] L. Findlater, B. Chinh, D. Jain, J. Froehlich, R. Kushalnagar, and A. C. Lin, "Deaf and hard-of-hearing individuals' preferences for wearable and mobile sound awareness technologies," in *Proc. CHI Conf. Human Factors in Computing Systems*, 2019.

[5] S. M. Goodman, S. Kirchner, R. Guttman, D. Jain, J. Froehlich, and L. Findlater, "Evaluating smartwatch-based sound feedback for deaf and hard-of-hearing users across contexts," in *Proc. CHI Conf. Human Factors in Computing Systems*, 2020.

[6] J. F. Gemmeke *et al.*, "Audio Set: An ontology and human-labeled dataset for audio events," in *Proc. IEEE ICASSP*, 2017, pp. 776–780.

[7] D. Jain *et al.*, "HomeSound: An iterative field deployment of an in-home sound awareness system for deaf or hard of hearing users," in *Proc. CHI Conf. Human Factors in Computing Systems*, 2020.

[8] D. Jain, H. Ngo, P. Patel, S. Goodman, L. Findlater, and J. Froehlich, "SoundWatch: Exploring smartwatch-based deep learning approaches to support sound awareness for deaf and hard of hearing users," in *Proc. ACM ASSETS*, 2020.

[9] D. Jain *et al.*, "ProtoSound: A personalized and scalable sound recognition system for deaf and hard-of-hearing users," in *Proc. CHI Conf. Human Factors in Computing Systems*, 2022.

[10] G. Laput, K. Ahuja, M. Goel, and C. Harrison, "Ubicoustics: Plug-and-play acoustic activity recognition," in *Proc. ACM UIST*, 2018.

[11] J. Z. Huang, H. Chhabria, and D. Jain, "'Not there yet': Feasibility and challenges of mobile sound recognition to support deaf and hard-of-hearing people," in *Proc. ACM ASSETS*, 2023, pp. 1–14.

[12] T. Matthews, J. Fong, F. W.-L. Ho-Ching, and J. Mankoff, "Evaluating non-speech sound visualizations for the deaf," *Behav. Inf. Technol.*, vol. 25, no. 4, pp. 333–351, 2006.

[13] J. Snell, K. Swersky, and R. Zemel, "Prototypical networks for few-shot learning," in *Proc. NeurIPS*, vol. 30, 2017.

[14] S. M. Goodman *et al.*, "Toward user-driven sound recognizer personalization with people who are d/Deaf or hard of hearing," *Proc. ACM Interact. Mob. Wearable Ubiquitous Technol.*, vol. 5, no. 2, 2021.

[15] S. Liu, Z. Zhou, J. Du, L. Shangguan, J. Han, and X. Wang, "UbiEar: Bringing location-independent sound awareness to the hard-of-hearing people with smartphones," *Proc. ACM Interact. Mob. Wearable Ubiquitous Technol.*, vol. 1, no. 2, 2017.

[16] M. Mielke and R. Brück, "A pilot study about the smartwatch as assistive device for deaf people," in *Proc. ACM ASSETS*, 2015, pp. 301–302.

[17] S. Hershey *et al.*, "CNN architectures for large-scale audio classification," in *Proc. IEEE ICASSP*, 2017, pp. 131–135.

[18] J. Cramer, H.-H. Wu, J. Salamon, and J. P. Bello, "Look, listen, and learn more: Design choices for deep audio embeddings," in *Proc. IEEE ICASSP*, 2019, pp. 3852–3856.

[19] Q. Kong, Y. Cao, T. Iqbal, Y. Wang, W. Wang, and M. D. Plumbley, "PANNs: Large-scale pretrained audio neural networks for audio pattern recognition," *IEEE/ACM Trans. Audio, Speech, Lang. Process.*, vol. 28, pp. 2880–2894, 2020.

[20] Y. Gong, Y.-A. Chung, and J. Glass, "AST: Audio spectrogram transformer," in *Proc. Interspeech*, 2021, pp. 571–575.

[21] M. Plakal and D. Ellis, "YAMNet," TensorFlow Model Garden, 2020. [Online]. Available: https://github.com/tensorflow/models/tree/master/research/audioset/yamnet

[22] A. G. Howard *et al.*, "MobileNets: Efficient convolutional neural networks for mobile vision applications," arXiv:1704.04861, 2017.

[23] J. Salamon and J. P. Bello, "Deep convolutional neural networks and data augmentation for environmental sound classification," *IEEE Signal Process. Lett.*, vol. 24, no. 3, pp. 279–283, 2017.

[24] A. Kumar and B. Raj, "Audio event detection using weakly labeled data," in *Proc. ACM Multimedia*, 2016, pp. 1038–1047.

[25] B. Jacob *et al.*, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE CVPR*, 2018, pp. 2704–2713.
[26] E. Fonseca, X. Favory, J. Pons, F. Font, and X. Serra, "FSD50K: An open dataset of human-labeled sound events," *IEEE/ACM Trans. Audio, Speech, Lang. Process.*, vol. 30, pp. 829–852, 2022.
[27] K. J. Piczak, "ESC: Dataset for environmental sound classification," in *Proc. ACM Multimedia*, 2015, pp. 1015–1018.
[28] N. Turpault, R. Serizel, A. Parag Shah, and J. Salamon, "Sound event detection in domestic environments with weakly labeled data and soundscape synthesis," in *Proc. Detection and Classification of Acoustic Scenes and Events Workshop (DCASE)*, 2019, pp. 253–257.
[29] A. Mesaros, T. Heittola, and T. Virtanen, "Metrics for polyphonic sound event detection," *Applied Sciences*, vol. 6, no. 6, p. 162, 2016.
[30] A. Mesaros, T. Heittola, T. Virtanen, and M. D. Plumbley, "Sound event detection: A tutorial," *IEEE Signal Process. Mag.*, vol. 38, no. 5, pp. 67–83, 2021.
[31] C. Elkan, "The foundations of cost-sensitive learning," in *Proc. IJCAI*, 2001, pp. 973–978.
[32] J. Lin, K. Kilgour, D. Roblek, and M. Sharifi, "Training keyword spotters with limited and synthesized speech data," in *Proc. IEEE ICASSP*, 2020, pp. 7474–7478.
[33] D. Scripka, "openWakeWord," GitHub, 2023. [Online]. Available: https://github.com/dscripka/openWakeWord
[34] L. Wan, Q. Wang, A. Papir, and I. Lopez Moreno, "Generalized end-to-end loss for speaker verification," in *Proc. IEEE ICASSP*, 2018, pp. 4879–4883.
[35] ISO 8201:2017, *Alarm systems—Audible emergency evacuation signal—Requirements*. Geneva: ISO, 2017.